Codebase Change Summary — Week 11

System Testing and Acceptance Testing
SE 413/513 — Software Testing

Demo Testing Suite

What changed in `demo-testing-suite/` since Week 10, and why

1 Purpose of This Document

The `demo-testing-suite/` codebase is a single, shared project that grows one lecture at a time. This document summarizes what was added or changed for Week 11 and explains *why* each change exists — which concept from the two Week 11 readings, *System Testing: Testing the Whole System from the Outside* and *Acceptance Testing: Building the Right Thing*, it demonstrates. Read it alongside the readings (`reading_materials/w11.1_system_testing.pdf`, `reading_materials/w11.2_acceptance_testing.pdf`) and `demos/system_acceptance_testing/README.md`.

Key Idea

Nothing from Weeks 1–10 was removed or changed in behavior. This week adds one self-contained project for both readings: Week 10's shop, grown into a system that runs on its own and is tested from the outside — as a separate server process, over HTTP only — at two new levels. System tests check it against its system requirements; acceptance tests, written in Gherkin, check it against its user stories. Writing the system tests found a real defect, present in Week 10's shop too.

2 What Changed At a Glance

File	Status	Purpose
The readings' system (not in the default run)		
`demos/system_acceptance_testing/shop/`	New	Week 10's shop plus `GET/health`, `GET/products`, coupon codes (`pricing.py`), stricter input validation, and a JSON `500` handler.
`shop/service_racy.py`, `shop/pricing_per_unit.py`	New	Two deliberately defective components: one oversells under concurrency; one misreads the coupon story.
`serve.py`, `variants.py`	New	Run the shop as one system (`-port 0`, `-db`, `-seed`); switch defective components in by environment variable, also inside the server process.
`fakes/payment_provider.py`	New (copy)	Week 10's fake PayFast, plus test-only `/_test/config` and `/_test/charges`.
`specs/`	New	System requirements, a one-page system test plan, user stories with acceptance criteria, the API contract v1.1, and the patch that reproduces finding F1.
`tests/conftest.py`	New	The system test harness: starts `serve.py` as a subprocess and talks to it over HTTP.
`tests/system/`	New	23 system tests: scenarios, hostile input, access control, concurrency, performance, recovery.
`tests/acceptance/`	New	3 `.feature` files, their step definitions, and 14 scenarios run by `pytest-bdd`.
`tests/unit/`, `tests/integration/`	New	The developer's pricing tests, and Week 10's integration harness carried over: the levels below, for comparison.
Tooling and documentation		
`requirements.txt`	Modified	Adds `pytest-bdd`.
`pytest.ini`	Modified	Registers the `system` and `acceptance` markers.
`docs/DEMO_GUIDE_week11_system_acceptance_testing.md`	New	Section-by-section lecture companion for both readings.
`README.md` (suite and repository)	Modified	Layout, the Week 11 entry, and the schedule.

Tip

Install the new dependency before running anything this week:

```
cd demo-testing-suite
pip install -r requirements.txt
```

3 The Readings' System, Runnable

Location: `demos/system_acceptance_testing/`. **Run its commands from inside the folder:**

```
cd demos/system_acceptance_testing
```

```
pytest                              # 58 passed, in about ten seconds
```

`demos/integration_testing/` is unchanged: the Week 10 reading quotes it line by line. The new folder started as a copy of it.

Folder	Level	How the shop runs	Tests
`tests/unit/`	unit (the developer's tests)	not at all	8
`tests/integration/`	integration (Week 10's)	inside the test process	13
`tests/system/`	system	separate process, HTTP only	23
`tests/acceptance/`	acceptance (Gherkin)	separate process, HTTP only	14

Two defective components, one switch each

```
SHOP_SERVICE=racy       pytest tests/system
SHOP_COUPONS=per_unit   pytest tests/acceptance
```

	unit	integration	system	acceptance
`SHOP_SERVICE=racy`	8 passed	13 passed	**1 failed**	14 passed
`SHOP_COUPONS=per_unit`	8 passed	13 passed	23 passed	**2 failed**

Each defect is caught at exactly one level. The *racy service* charges the card before taking the stock, from a value it read before the charge: with ten simultaneous customers it sells ten of five mugs and leaves the stock at four. Only a test that runs the real, multi-threaded server can see it. The *per-unit pricing* reads "FLAT5: $5 off orders over $10" as "$5 off each unit over $10". The developer's own tests order one unit, where both readings agree; only the examples agreed with the product owner (two mugs; four stickers) catch it.

4 A Real Finding: Hostile Input (F1)

When the system tests were first run against Week 10's input validation, two hostile requests — a `sku` sent as a JSON list, and a `quantity` of 10^{30} — crashed the request inside SQLite, and Flask answered with its default *HTML* `500` page. That breaks the shop's own API contract ("every error response is JSON… never an HTML page"). It is present in `demos/integration_testing/` as well; that folder is left as the Week 10 reading printed it.

The fix needed a requirement that did not exist: the product owner set a maximum of 100 units per order. `validate_order()` now checks that `sku` is a string and `quantity` is 1–100, and a JSON handler covers any `500`. To see the failure again:

```
patch -p1 < specs/f1_week10_validation.patch
pytest tests/system/test_robustness_and_security.py     # 2 failed
patch -R -p1 < specs/f1_week10_validation.patch
```

5 Updated: Expected Test Results

From the project root:

```
cd demo-testing-suite
pip install -r requirements.txt
```

```
pytest -v
```

you should still see **75 passed, 3 xfailed, 1 failed**, exactly Week 10's result: nothing in `app/` or `tests/` changed this week. `demos/system_acceptance_testing/` is not part of that run (`pytest.ini` sets `testpaths = tests`); run it from inside its folder.

Common Pitfall

Why the main suite gets no system tests this week. `app/` is a library of modules, not a program that runs on its own: there is no system to start. System and acceptance testing need a deployable system, which is why both readings use the shop. Your semester project, which is a runnable system, is where you apply them.

6 How This Connects to the Readings

Key Idea

Weeks 2–10 tested the system's parts and the promises between them. Week 11 tests the whole: first against the team's specification, as a customer, operator, or attacker would meet it (system testing); then against what the customer actually meant (acceptance testing). Each level found a defect every level below it passed.

7 Suggested Next Steps

1. Run the switch table above yourself, one cell at a time, and *predict* each result first.
2. Read `shop/service_racy.py` and find the defect before running the system tests against it.
3. Before reading `specs/user_stories.md`'s three-amigos notes, write down what you think 2 mugs with FLAT5 should cost.
4. Implement restocking (System Testing, Exercise 6) and order history (Acceptance Testing, Exercise 6), tests first.
5. Write a system test plan and three Gherkin scenarios for your semester project.